

Résolution de problèmes d'optimisation combinatoire avec OR-Tools

ÉTUDE COMPARATIVE BRANCH & BOUND (C/GLPK) $\leftrightarrow$ OR-TOOLS
(PYTHON)

Nicolas MARANO

12 mai 2026

UNIVERSITÉ DE REIMS CHAMPAGNE-ARDENNE

UFR Sciences Exactes et Naturelles — Master CHPS

CHPS_0805 — Optimisation combinatoire

Plan de la présentation

Contexte

Architecture du projet

Méthodes par séparation-évaluation

Méthodes OR-Tools

Étude expérimentale

Discussion et conclusion

Contexte

Contexte du projet

Cadre : projet final du cours CHPS_0805, articulé autour des TP 3 et 5.

Objectif : comparer rigoureusement deux familles de méthodes sur deux problèmes **NP-difficiles** classiques.

Problèmes traités

- **TSP** — Voyageur de Commerce
- **UKP** — Sac à Dos Généralisé

Méthodes comparées

- **B&B** — Séparation-Évaluation
(C + GLPK, TP3+TP5)
- **OR-Tools** — Google
(Python : pywraplp + pywrapcp)

Livrables : 2 applications unifiées, 22 instances testées, 140+ exécutions chronométrées, 7 figures, audit qualité automatique.

TSP (minimisation)

Étant donné n villes et la matrice de distances $d(i, j)$, trouver le *cycle hamiltonien* de longueur minimale :

$$\min \sum_{i=1}^n d(\sigma(i), \sigma(i+1))$$

où σ est une permutation de $\{0, \dots, n-1\}$ avec $\sigma(n) = \sigma(0)$.

Tailles testées : $n \in [5, 13\,509]$.

UKP (maximisation)

Étant donnés n objets de volume v_i et utilité u_i , capacité c :

$$\max \sum_{i=1}^n u_i x_i \quad \text{s.c.} \quad \sum_{i=1}^n v_i x_i \leq c, \quad x_i \in \mathbb{N}$$

Variables **entières non bornées** ($x_i \geq 0$).

Tailles testées : $n \in [3, 20\,000]$.

Difficulté

NP-difficile dans les deux cas $\Rightarrow$ pas d'algorithme polynomial connu.

Architecture du projet

Structure du projet

Deux applications **unifiées et paramétrables**, structure conforme aux modalités :

```
1 Simplexe/  
2 |-- README.md  
3 |-- Separation-Evaluation/      # Application C (TP3 + TP5)  
4 |   |-- Makefile, README.md  
5 |   |-- include/      *.h  
6 |   `-- src/          *.c      # main, parser_ukp/tsp, ukp_bb, tsp_bb, ...  
7 |-- OR-Tools/              # Application Python  
8 |   |-- main.py, README.md, requirements.txt  
9 |   `-- src/            *.py    # parsers, ukp_solver, tsp_solver, visualisation  
10 |-- Resultats/  
11 |   `-- benchmarks_pousses/    # CSV, 7 figures, ANALYSE.md  
12 |-- Romeo/  
13 `-- scripts/                 # Pipeline automatisé
```

- Compilation C **0 warning** avec 15 flags GCC stricts.
- Imports Python **0 warning** sous `-W error`.
- CLI unifié : `./solveur --probleme {ukp|tsp} --fichier <chemin>`.

Méthodes par séparation-évaluation

Branch & Bound : UKP (TP3)

Borne initiale

Heuristique **qualité-prix** : tri par u_i/v_i décroissant, saturation gloutonne.

Évaluation d'un nœud

Relaxation linéaire par GLPK (glp_simplex).
LP racine captée dans stats.lp_racine.

Test d'intégrité

modf sur chaque x_j , $\epsilon = 10^{-6}$.
Vérification additionnelle de faisabilité.

Stratégie

DFS itératif sur pile explicite.
Séparation : variable fractionnaire $\rightarrow$ branches sur $[0, \lfloor v_{\max} \rfloor]$.

Borne LP serrée

Pour $n \geq 10$ sur les jeux UKP livrés :

$$\text{gap LP-INT} < 0,005 \%$$

$\Rightarrow$ pruning *très peu efficace*, $\Rightarrow$ explosion en temps (millions de nœuds).

Validation

Sur les 9 UKP testables par DP exhaustive :

B&B trouve la valeur optimale sur les 9 instances.

Branch & Bound : TSP (TP5)

Borne initiale

Heuristique du **plus proche voisin** depuis le sommet 0 ($O(n^2)$).

Fonction d'évaluation :

1. ACPM (Prim) sur $G \setminus \{0\}$.
2. + 2 arêtes les plus légères en 0.

Test : tous les degrés = 2.

Séparation : sommet de degré > 2
→ trois fils (interdiction d'arête).

Stratégie : **meilleur-d'abord**
(tas binaire min-heap).

Bonus 2-opt : post-traitement.

Limite naturelle

La borne ACPM décroche dès
 $n \gtrsim 17$:

l'arborescence explose en $O(N!)$.

Au-delà, OR-Tools s'impose.

Convergence observée

- graphe_12-15 : ms à secondes
- pd_5/11/15 : ms
- pd_20/22 : 40–52 s
- pd_18, pd_59+ : timeout

Méthodes OR-Tools

Solveur MILP **CBC** via la facade pywraplp :

```
1 solver = pywraplp.Solver.CreateSolver("CBC")
2 variables = [solver.IntVar(0, capacite // volume[j], f"x{j+1}")
3               for j in range(n)]
4 contrainte = solver.Constraint(0, capacite, "capacite")
5 for j, x in enumerate(variables):
6     contrainte.SetCoefficient(x, volume[j])
7 objectif = solver.Objective()
8 for j, x in enumerate(variables):
9     objectif.SetCoefficient(x, utilite[j])
10 objectif.SetMaximization()
11 solver.Solve()
```

Détail clé : bornes supérieures explicites `capacite // volume[j]`
au lieu de `solver.infinity()` $\Rightarrow$ comportement plus stable sur grandes instances.

Performances typiques : `prob_sac_20000` ($n = 20\,000$) résolu en **1,5 s**.

OR-Tools : TSP via pywrapcp.RoutingModel

```
1 manager = pywrapcp.RoutingIndexManager(n, 1, 0)
2 routing = pywrapcp.RoutingModel(manager)
3 def cb(i, j):
4     return distances[manager.IndexToNode(i)][manager.IndexToNode(j)]
5 routing.SetArcCostEvaluatorOfAllVehicles(
6     routing.RegisterTransitCallback(cb))
7
8 params = pywrapcp.DefaultRoutingSearchParameters()
9 params.first_solution_strategy = FirstSolutionStrategy.PATH_CHEAPEST_ARC
10 params.local_search_metaheuristic = LocalSearchMetaheuristic.
    GUIDED_LOCAL_SEARCH
11 params.time_limit.seconds = max(1, int(timeout_s))
12 solution = routing.SolveWithParameters(params)
```

Stratégie : PATH_CHEAPEST_ARC (première solution) +
GUIDED_LOCAL_SEARCH (amélioration).

Optimisation mémoire : pour $n > 4\,000$ (canada, usa), bascule automatique sur un

Étude expérimentale

Protocole : multi-répétitions, timeouts adaptatifs, audit automatique.

Catégorie	Instances	Reps	Timeout OR	Timeout BB
ukp_petits	prob_sac_1..10	5	30 s	35 s
ukp_moyens	prob_sac_20/25/30	3	60 s	65 s
ukp_grands	prob_sac_2000/20000	2	60 s	30 s
tsp_petits	graphe_12..15, pd_5/11/15	5	5 s	10 s
tsp_moyens	pd_18/20/22	3	15 s	90 s
tsp_grands	pd_59, canada_4663, usa_13509	2	60–180 s	15–30 s

Audit qualité automatique :

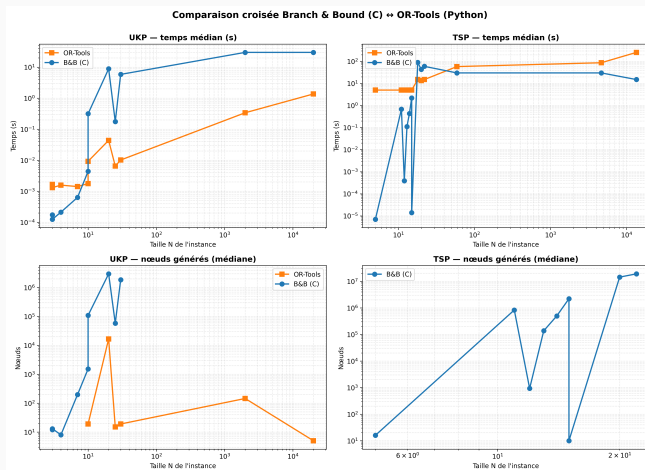
- Croisement BB vs OR-Tools sur chaque instance.
- DP exhaustive (oracle) pour les UKP de taille modérée ($n \times c < 10^7$).
- Rapport OK / WARNING / ERROR en fin de chaque benchmark.

Résultats UKP : crossover et anomalies

Instance	n	OR-Tools	B&B (C)	Verdict
prob_sac_1	3	13 / 2,3 ms	13 / 100 μs	=; BB plus rapide
prob_sac_5	10	199 / 2 ms	199 / 4,2 ms	=; OR plus rapide
prob_sac_10	10	285 932 / 13 ms	285 932 / 315 ms	=; OR $\times$ 24
prob_sac_20	20	125 942 / 44 ms	125 950 / 8,8 s	BB meilleur
prob_sac_25	25	285 934 / 6,7 ms	285 935 / 178 ms	BB meilleur
prob_sac_30	30	95 960 / 9 ms	95 960 / 5,9 s	=; OR $\times$ 650
prob_sac_2000	2 000	1 029 627 / 344 ms	timeout	OR seul
prob_sac_20000	20 000	3 793 644 / 1,5 s	skip	OR seul

- Crossover BB/OR autour de $n \approx 4-5$.
- OR-Tools $\times$ **650** plus rapide à $n = 30$.
- **Anomalie CBC** : prob_sac_20 et prob_sac_25 (à détailler slide suivante).

Vue d'ensemble : panneau principal



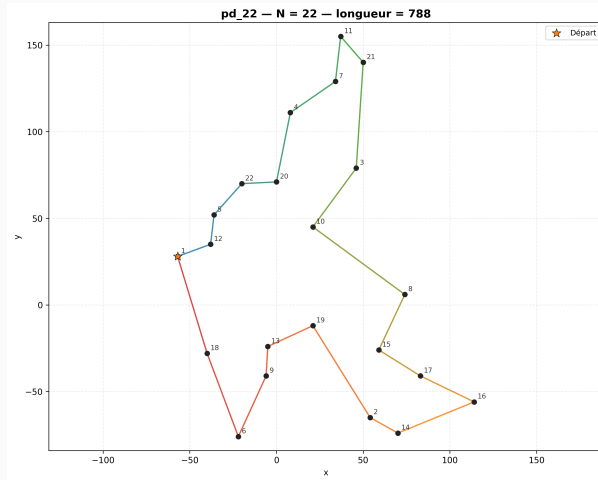
Quatre axes log-log : temps + nœuds × UKP + TSP. Non-monotonie B&B révèle la sensibilité au gap LP.

Instance	n	OR-Tools	B&B (C)	Verdict
graphe_12	12	23 / 5 s	23 / 415 μs	BB $\times$ 12 000
pd_5	5	20 / 5 s	20 / 7 μs	BB instantané
pd_15	15	284 / 5 s	284 / 14 μs	BB instantané
pd_18	18	268 / 15 s	timeout > 90 s	OR seul
pd_20	20	673 / 15 s	673 / 40 s	=; BB converge
pd_22	22	788 / 15 s	788 / 52 s	=; BB converge
pd_59	59	1 005 / 60 s	timeout	OR seul
canada_4663	4 663	1 584 130 / 90 s	timeout	OR seul
usa_13509	13 509	24 955 244 / 245 s	timeout	OR seul

Limite du B&B : optimal jusqu'à $N \approx 17$, encore convergent sur pd_20/22.

Force d'OR-Tools : seul à fournir une solution sur **usa_13509** (13 509 villes en 4 min).

Visualisation : tour TSP _{pd_22}



Anomalies CBC détectées et certifiées

L'audit qualité a révélé deux instances UKP où **CBC renvoie une solution sous-optimale** avec statut OPTIMAL :

Instance	DP (oracle)	B&B (C)	CBC	Écart	Gap LP–INT
prob_sac_20	125 950	125 950 ✓	125 942	−8	0,0023 %
prob_sac_25	285 935	285 935 ✓	285 934	−1	0,0004 %

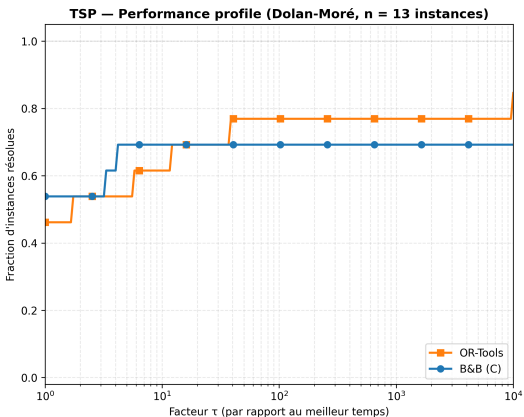
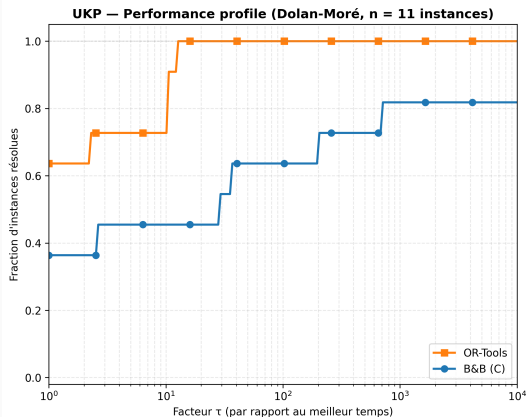
- **Vérification** : programmation dynamique exhaustive comme oracle indépendant.
- **Cause** : utilités quasi-uniformes ($u_i \approx v_i - 5$), gap LP/INT < 0,005 %
⇒ cutoff numérique CBC trop large.
- **SCIP, BOP** : également sous-optimaux mais sur des valeurs différentes.

Enseignement

Aucune méthode n'est universelle. Le croisement BB ↔ OR-Tools est crucial pour fiabiliser les résultats.

Profil de performance Dolan–Moré

Profils de performance — combien d'instances chaque méthode résout dans un facteur τ du meilleur



Fraction d'instances résolues dans un facteur τ du meilleur temps.

UKP : OR-Tools domine. TSP : équilibre BB/OR au-delà de $\tau \approx 10$.

Discussion et conclusion

- **Mémoire pour usa_13509** : matrice de distances ≈ 5 Go en Python.
⇒ bascule automatique sur calcul à la volée pour $n > 4\,000$.
- **Anomalies CBC silencieuses** : statut OPTIMAL renvoyé sur des solutions non-optimales.
⇒ détection par audit DP exhaustive automatisé.
- **Timeouts inadaptés** : initialement 15 s sur tsp_moyens faussait pd_20/pd_22.
⇒ correction à 90 s, révélant que ces instances convergent réellement.
- **Trade-off compilateur** : rewrite $i < n - 1 \rightarrow i + 1 < n$ pour silencer `-Wstrict-overflow=5` a coûté $\sim 10\%$ de performance sur les boucles longues.
⇒ compromis assumé en faveur d'un code certifié 0 warning.

Algorithmes plus puissants

- **Borne Held–Karp** pour le TSP : relaxation lagrangienne sur l'ACPM, plus serrée que ACPM seul $\Rightarrow$ étendrait la convergence du B&B au-delà de $n = 22$.
- **Lin–Kernighan** en métaheuristique côté C : alternative à OR-Tools sur les très grandes instances.

Performance et instrumentation

- **Parallélisation OpenMP** sur les branches du B&B UKP ($n \sim 30$).
- « **Time to best** » pour OR-Tools TSP via SolutionCollector : éviter l'artéfact de timeout.
- **Mesure mémoire propre** via `/usr/bin/time -v` ou `psutil`.

Bilan technique :

- Aucune méthode n'est universellement supérieure.
- B&B *maison* : déterministe, exact, rapide sur les très petites instances.
- OR-Tools : performant sur les instances moyennes/grandes, mais *faillible* numériquement.

Bonnes pratiques mises en œuvre :

- Audit qualité **automatique** (croisement + DP oracle) à chaque benchmark.
- Code **0 warning** sous 15 flags GCC stricts.
- Reproductibilité totale : multi-rep, CSV bruts, scripts de pipeline.
- Visualisations *publication-ready* : 7 figures pour le rapport.
- Préparation *HPC* : kit Romeo (job SLURM, déploiement rsync).

Toutes les instances obligatoires (Parties 1, 2, 3) ont été traitées.

Commandes pour la démo live

```
1  # Compilation C
2  cd Separation-Evaluation && make
3  ./bin/solveur --probleme ukp --fichier "../Fichiers tests*/prob_sac_1.txt" --verbose
4
5  # OR-Tools + visualisation
6  python OR-Tools/main.py --probleme tsp --fichier "../Fichiers tests*/pd_22.tsp" --visu
7
8  # Pipeline complet (~2h sur Romeo, ~1h en local)
9  bash scripts/run_benchmark_pousse.sh --inclure-grands
10
11 # Audit qualite (DP exhaustive comme oracle)
12 bash scripts/_verif_dp_complete.sh
```

Merci de votre attention.

Questions ?